

ACX City

Optimal Audiobook Builder, Background Adder, and Audio Remastering Platform

Full product plan, technical specification, delivery roadmap, and AI-agent execution blueprint

Version 1.0 | Prepared July 28, 2026 | Repository baseline: redinc23/acx-city @ f9a81e54



Approved product board used as the visual foundation for this plan

Core decision: Do not replace the durable Flask/Postgres/worker foundation. Expand it into a project-based, non-destructive audio production system with versioned manuscripts, multi-step renders, timeline mixing, standards-aware mastering, and publish-ready export packages.

Document Control

Field	Value
Product	ACX City audiobook production platform
Primary repository	github.com/redinc23/acx-city
Repository baseline	main branch at commit f9a81e54eb7ac8985858b2ca4380e108b0b56d85
Plan type	Product requirements + UI specification + technical implementation plan
Primary users	Independent authors, publishers, agencies, businesses, narrators, audio engineers
Target releases	Builder MVP, Pro Production Beta, General Availability
Planning horizon	28 weeks with a cross-functional team; longer for a single developer or agent
Source of truth rule	Repository reality wins over assumptions; this document defines the target state

Contents

- 1. Executive Summary and Product Decision
- 2. Current Repository Baseline and Gap Analysis
- 3. Product Vision, Principles, Personas, and Scope
- 4. Target Information Architecture and End-to-End Workflow
- 5. Complete Feature Requirements
- 6. UI and Interaction Specification
- 7. Target Technical Architecture
- 8. Data Model and API Plan
- 9. Exact Repository File Plan
- 10. Delivery Roadmap and Pull Request Sequence
- 11. Testing, Security, Operations, and Release Management
- 12. Metrics, Risks, Governance, and Definition of Done
- 13. First 30 Days: Immediate Execution Plan
- Appendices: Priority Matrix, Event Taxonomy, Screen States, and Sources

1. Executive Summary and Product Decision

ACX City already has the difficult infrastructure fundamentals: a restart-safe Postgres-backed queue, per-chapter checkpoints, organization-scoped authorization, pluggable TTS providers, content-addressed caching, storage abstraction, quotas, rate limits, QC gating, retention, structured logs, and deployable API, worker, frontend, dashboard, and MCP services. The correct strategy is not a rewrite. The correct strategy is to transform the current single-run synthesis interface into a persistent production workspace.

The target product is a non-destructive audiobook digital audio workstation optimized for books rather than music. It must support manuscript intelligence, chapter editing, voice casting, selective regeneration, uploaded human narration, background music and sound effects, automated and manual mastering, standards-aware QC, review and approvals, version history, and export packages for multiple destinations.

Most important product correction: The current name and positioning can imply direct ACX acceptability, but ACX currently states that unauthorized text-to-speech, AI, or automated recordings are prohibited unless authorized. The platform must therefore present ACX technical validation as a standards profile, not as a guarantee of marketplace acceptance, and must capture rights, voice-consent, and distribution authorization.

1.1 Outcomes

Outcome	Target capability	Success signal
Create	Turn a manuscript or uploaded narration into a structured, editable audiobook project	A new user can create a valid project and preview the first chapter in under 10 minutes
Control	Edit chapters, pronunciations, voices, pacing, pauses, and selected passages without rebuilding the entire book	At least 90% of corrections regenerate only affected segments
Enhance	Mix licensed music, ambience, and effects with narration-aware ducking	Mixes remain intelligible and reversible, with per-chapter overrides
Master	Analyze and remaster narration using one-click profiles and manual controls	Every export receives a reproducible report and pass/fail explanation
Review	Collect time-coded comments and approvals from authors, editors, and clients	Reviewers can comment without downloading raw project files
Publish	Generate chapter files, M4B, MP3, samples, credits, metadata, and manifests	A publish package contains every expected file with deterministic names and checksums
Operate	Scale safely across organizations, providers, workers, and storage	No cross-tenant access, no lost jobs, and measurable cost per project

1.2 Release sequence

Release	Timing	Definition
Builder MVP	Weeks 1-10	Stable project workspace, chapter editor, corrected job status model, voice presets, pronunciation rules, selective builds, chapter preview, frontend tests

Release	Timing	Definition
Pro Production Beta	Weeks 11-20	Timeline editor, uploaded narration, background mixer, remastering, enhanced QC, review links, versioned renders
General Availability	Weeks 21-28	Publish packages, billing, analytics, organization collaboration, webhooks, scoped MCP/API tools, scale and security hardening

2. Current Repository Baseline and Gap Analysis

2.1 What exists and should be preserved

Area	Current implementation	Plan decision
API	Flask API with authenticated, organization-scoped endpoints	Preserve. Introduce versioned /api/v1 modules rather than a rewrite.
Queue	Postgres rows claimed with FOR UPDATE SKIP LOCKED, retries, lease heartbeat, orphan recovery	Preserve as the core queue. Extend with job types and durable job steps.
Workers	Standalone worker executes a chapter-by-chapter pipeline and checkpoints progress	Preserve. Split pipeline steps into idempotent handlers.
Data	Organizations, users, memberships, projects, jobs, chapter results, attempts, usage events	Extend into a persistent production domain rather than replacing.
TTS	Provider abstraction with Edge and AWS Polly	Expand provider capability metadata, SSML support, preview, style and locale support.
Audio	FFmpeg and pydub assembly, normalization, MP3 and M4B export	Retain FFmpeg, replace ad hoc mastering with profile-driven processing and measurements.
Storage	Local and S3-compatible storage with signed URLs	Make object storage mandatory for production and store all source, proxy, stem, and export artifacts.
Frontend	React/Vite single-page producer interface	Evolve into routed project workspace with a shared design system.
Ops	Next.js dashboard for queue, usage, QC and provider health	Keep as operator console; add worker, storage, delivery, cost, and audit views.
Automation	Read-only MCP server	Add organization-scoped write tools only after scoped authentication and audit logging.

2.2 Verified implementation strengths

- **Durability:** Jobs are database records, workers claim them safely, and per-chapter state is committed so restarts can resume.
- **Tenant isolation:** Projects and jobs belong to organizations and access is checked through membership rather than possession of a task identifier.

- **Operational controls:** The repository already includes quota checks, rate limiting, usage costs, QC gating, retention sweeps, structured logging, and optional Sentry.
- **Provider flexibility:** The provider contract isolates vendor SDKs and supports a free Edge path plus an AWS Polly fallback.
- **Portable storage:** Object keys are namespaced by organization and job, and clients receive expiring signed URLs.
- **Agent readiness:** An MCP server already exposes health, jobs, organizations, and usage for operator agents.

2.3 Critical gaps and contradictions to resolve first

Priority	Finding	Impact	Required action
P0	Frontend status types expect started, processing, completed, while backend emits queued, running, succeeded, needs_review, failed, canceled.	Completed jobs can appear permanently active and download actions may never render.	Create one shared status contract and fix App.tsx, types, ProgressTracker, schemas, and tests before new UI work.
P0	The current UI creates a new Project for every synthesis request and offers no reusable project workspace.	Users cannot revise, compare, resume, or manage a book as a durable product.	Add project CRUD, manuscript revisions, chapters, drafts, and render history.
P0	Current audio QC uses dBFS normalization and simple silence/clipping checks.	It does not fully validate destination requirements such as ACX RMS, peak, noise floor, bitrate, sample rate, file duration, naming, credits, or channel consistency.	Build a versioned QC profile system using fprobe/FFmpeg analysis and explicit destination reports.
P0	ACX currently prohibits unauthorized TTS or automated recordings unless authorized.	Branding or exports could incorrectly imply submission eligibility.	Add distribution warnings, rights attestation, voice consent, and a technical-only compatibility statement.
P0	Frontend has a Vitest script but CI does not execute frontend tests; the UI test gate is effectively absent.	UI regressions can ship despite backend quality.	Add React Testing Library unit tests and Playwright end-to-end tests; make them required CI checks.
P1	Uploads are extracted to text and the temporary file is deleted.	Original manuscript provenance, replacement, and reproducibility are lost.	Persist original source files and normalized text as versioned assets in object storage.
P1	Only book-wide provider and voice settings are modeled.	No per-chapter or character voice casting, pronunciation rules, or selective regeneration.	Add voice profiles, assignments, pronunciation lexicons, segment render hashes, and revision-aware caching.
P1	No audio timeline, background mixing, uploaded narration, or mastering chain exists.	The board's main differentiators are absent.	Introduce non-destructive timeline, tracks, clips, mix presets, and remaster presets.
P1	The jobs list is limited to 100 without cursor pagination or filters.	Organizations with production volume cannot manage history effectively.	Add cursor pagination, search, status filters, project filters, and archival views.
P1	The cache statistics endpoint is public and a CORS override permits wildcard origins with credentials.	Operational information can leak and a configuration mistake can weaken browser security.	Require admin authorization for cache stats and reject wildcard credentialed CORS in production.
P1	The Next.js dashboard CI uses npm install instead of npm ci.	Dashboard dependency resolution is not fully deterministic.	Use npm ci and execute dashboard tests in CI.
P2	MCP uses one bearer key and exposes cross-organization operator reads.	Write tools would be unsafe without stronger scopes.	Issue scoped service tokens, enforce organization claims, add tool-level permissions, and audit each invocation.

2.4 Current product maturity score

Dimension	Current	Target GA	Comment
Infrastructure durability	8/10	9/10	Strong foundation; needs step-level orchestration and artifact lineage.
Security and tenancy	8/10	9/10	Strong base; needs fine-grained permissions, audit events, scoped agents, and CORS hardening.
Audiobook creation UX	3/10	9/10	Current UI is a single form rather than a persistent book workspace.
Audio editing and mixing	1/10	8/10	Assembly exists, but no non-destructive editor or background mixer.
Mastering and compliance	3/10	9/10	Basic checks exist; standards profiles and full reports are needed.
Collaboration	1/10	8/10	No comments, approvals, review links, or version comparisons.
Publishing readiness	4/10	9/10	M4B/MP3 exist; chapter packages, credits, samples, metadata, and manifests are missing.
Test safety	7/10 backend / 1/10 frontend	9/10	Backend is strong; frontend and dashboard need meaningful coverage.

3. Product Vision, Principles, Personas, and Scope

3.1 Product vision

ACX City will be the book-first production environment where a user can move from manuscript to polished, reviewable, publish-ready audio without learning a traditional music DAW. Every transformation remains traceable, reversible, reproducible, and scoped to the organization that owns the project.

3.2 Product principles

Principle	Product rule
Book-first, not track-first	Chapters, sections, speakers, pronunciation, credits, samples, and destination packages are first-class objects.
Non-destructive by default	Never overwrite source text or source audio. Store edit decisions, presets, and derived artifacts separately.
Selective rebuilds	A correction to one sentence should regenerate and reassemble only the affected segments and outputs.
Explain every result	QC failures, estimated costs, provider decisions, and remaster changes must be visible and auditable.

Principle	Product rule
Human review remains central	Automation accelerates production but approval gates remain explicit for script, voice, mix, master, and publish stages.
Provider and destination neutrality	TTS and export standards are adapters, not hard-coded assumptions.
Rights-aware media	Every voice, music track, sound effect, and source manuscript can carry provenance, consent, and license metadata.
Accessible and keyboard-operable	Core workflows must work with keyboard navigation, captions, semantic controls, and readable contrast.

3.3 Primary personas

Persona	Primary goal	Essential capabilities
Independent author	Produce a professional audiobook without an audio engineering team	Guided wizard, low-cost voices, one-click mastering, clear compliance report, simple review links
Publisher production editor	Manage many titles, people, deadlines, costs, and quality gates	Templates, batch operations, roles, approvals, audit logs, analytics, reusable assets
Narrator or producer	Record, edit, repair, master, and deliver human narration	Audio upload, pickups, waveform editor, remaster chain, time-coded comments, chapter exports
Agency or studio	Deliver branded, white-label production to clients	Multiple organizations, client review portals, custom presets, usage reports, API and webhooks
Business learning team	Create narrated training and internal knowledge audio	Fast updates, multiple languages, brand music, access control, version history
Operations administrator	Keep the platform reliable, secure, and cost controlled	Queue health, worker capacity, provider status, storage, quotas, retention, incidents

3.4 In-scope and non-goals

In scope for GA	Not a GA requirement
Manuscript-to-audio production, uploaded human narration, multi-voice projects, chapter editing, background mixing, remastering, standards-aware QC, review, export, team access, billing, API and MCP	A full music composition DAW, live multi-user waveform editing, a narrator labor marketplace, direct automated submission into every retailer, unrestricted voice cloning, or replacing professional legal review

4. Target Information Architecture and End-to-End Workflow

4.1 Primary navigation

Navigation item	Purpose	Key views
Dashboard	Portfolio status and recommended next actions	Recent projects, blocked reviews, active jobs, spend, QC trends
Projects	Persistent book workspaces	Grid/list, filters, archive, templates, create/import
Jobs	All asynchronous work	Queue, retries, cancel, logs, estimates, failures
Library	Reusable voices, music, effects, presets, pronunciations, covers	Search, license metadata, usage history, favorites
Reviews	Items awaiting user or client decisions	Script approval, voice samples, mix review, master review, publish approval
Analytics	Usage, costs, quality, throughput, provider and cache efficiency	Organization and project reports
Team	Members, roles, invitations, service accounts	Access management and audit history
Settings	Organization, billing, storage, providers, notifications, integrations	Policy and configuration

4.2 Project workspace stages

Stage	User action	System output	Approval gate
1. Import	Upload manuscript, paste text, connect document, or upload narration	Source asset, normalized text, extraction report	Source accepted
2. Script and chapters	Review structure, edit speakable text, remove non-narrated matter	Versioned chapters and segment map	Script locked for render
3. Voices	Select provider, narrator profile, character voices, pronunciation rules	Voice assignments and preview clips	Voice cast approved
4. Build	Estimate cost and start selected chapters or full book	Narration stems and render manifest	Narration accepted
5. Edit and background	Trim, regenerate, arrange music/ambience/effects, set ducking	Non-destructive timeline and mix preview	Mix approved
6. Remaster	Apply one-click profile or manual chain, compare A/B	Mastered chapter artifacts and processing report	Master approved
7. QC	Run destination profile and resolve findings	Pass/fail report, exceptions, waivers	QC approved
8. Publish	Complete credits, sample, metadata, cover, file naming, exports	Versioned publish package and manifest	Package approved

4.3 Status model

Use one canonical vocabulary across backend, web app, dashboard, API, webhooks, and MCP. Legacy endpoints may translate during deprecation, but UI types must never invent alternate states.

Object	Statuses
Project	draft, active, paused, completed, archived
Revision	draft, approved, superseded
Job	queued, running, needs_review, succeeded, failed, canceled
Job step	pending, ready, running, blocked, succeeded, failed, skipped, canceled
Render	draft, rendering, review, approved, rejected, obsolete
Export package	draft, validating, ready, failed, delivered, revoked
Review	open, changes_requested, approved, resolved

5. Complete Feature Requirements

The following catalog defines the optimal product. Each capability is assigned a release priority: P0 is required for stability or compliance, P1 for the core marketable product, P2 for professional depth, and P3 for ecosystem expansion.

5.1 Project and workspace management

Priority	Capability	Requirement
P0	Persistent projects	Create, rename, duplicate, archive, restore, and delete a book workspace without creating a new project for each render.
P0	Autosave and drafts	Save every text and settings change with optimistic concurrency and visible saved/saving/error states.
P0	Revision history	Create immutable manuscript revisions and restore or compare prior versions.
P1	Templates	Save organization templates for genres, voice, pacing, mix, mastering, QC, and exports.
P1	Project checklist	Show stage completion, blockers, approvals, and required metadata.
P1	Bulk operations	Archive, tag, assign, export, and rerun QC across selected projects.
P2	Series management	Group books into a series and share credits, voice casts, pronunciations, and artwork rules.
P2	Client workspaces	Agency view with client branding, review permissions, and isolated assets.

5.2 Manuscript ingestion and intelligence

Priority	Capability	Requirement
P0	Source formats	Retain DOCX, TXT, and PDF support and add EPUB and Markdown through pluggable ingesters.
P0	Source preservation	Store the original file, checksum, MIME type, extraction version, and normalized text.
P0	Chapter editor	Add, remove, split, merge, rename, reorder, and mark front or back matter.
P0	Speakable text layer	Maintain a narration script separate from displayed source text so citations, footnotes, tables, URLs, and abbreviations can be transformed safely.
P1	Structure confidence	Show why a chapter was detected and allow one-click correction.
P1	Text linting	Detect empty chapters, repeated headers, page numbers, OCR artifacts, unbalanced punctuation, very long sentences, and unsupported symbols.
P1	Pronunciation candidates	Flag names, acronyms, foreign phrases, numbers, dates, equations, and URLs for review.
P2	Scanned PDF OCR	Optional OCR pipeline with page-level confidence and manual review.
P2	Document sync	Import or refresh from Google Drive and preserve a diff before replacing the current revision.

5.3 Script, pronunciation, and performance direction

Priority	Capability	Requirement
P0	Pronunciation dictionary	Organization, project, character, and chapter scopes with plain replacement, phoneme, or provider-specific entries.
P0	Pause and pacing controls	Book, chapter, paragraph, and selection overrides.
P1	Selection preview	Render selected text immediately without creating a full job.
P1	Regenerate selection	Replace only the affected segment and invalidate dependent mix/master artifacts.
P1	Performance notes	Attach style, emotion, emphasis, speed, pitch, and narrator direction where supported.
P2	Text-audio alignment	Store word or sentence timestamps for navigation, comments, and pickup replacement.
P2	Character glossary	Detect recurring speakers or character names and map them to voices and pronunciations.

5.4 Voice studio and casting

Priority	Capability	Requirement
P0	Voice catalog	Filter by provider, language, locale, gender, style, cost, sample availability, and commercial-use status.
P0	Voice preview	Use a standardized sample plus a project-specific excerpt, cached by voice and settings.
P0	Voice profiles	Save provider, voice, engine, speaking rate, pitch, style, and fallback.
P1	Per-chapter assignment	Override the default narrator for any chapter.
P1	Multi-voice casting	Assign voices to characters or marked dialogue segments.
P1	Fallback rules	Fail over to an approved voice or provider when a provider is unavailable.
P2	Human narrator profiles	Associate uploaded narration, pickup sessions, contracts, and sample reels.
P3	Licensed custom voices	Support verified-consent custom voices only, with consent receipts, allowed uses, expiration, and revocation.

5.5 Audiobook builder and orchestration

Priority	Capability	Requirement
P0	Preflight estimate	Calculate characters, expected audio duration, provider cost, cached reuse, and estimated completion time before launch.
P0	Build scope	Full book, selected chapters, changed chapters, failed chapters, or selected segments.
P0	Durable job steps	Extract, normalize, segment, synthesize or ingest, assemble, align, mix, master, QC, and export as checkpointed steps.
P0	Resume and retry	Resume at the failed step and preserve successful artifacts.
P1	Priority queue	Organization priority and plan-based concurrency without violating fairness.
P1	Real-time progress	Use server-sent events or WebSocket updates with polling fallback.
P1	Diff-aware cache	Hash normalized segments and settings so unchanged speech and downstream artifacts are reused safely.
P2	Scheduled builds	Queue large builds for a selected window and notify on completion or review.

5.6 Audio timeline and narration editor

Priority	Capability	Requirement
P1	Book-first timeline	Chapter lanes with waveform, transcript, markers, and selection playback.
P1	Non-destructive edits	Trim, split, move, mute, gain, fades, crossfades, and replace without altering source assets.
P1	Pickups	Mark errors, record or upload replacement audio, align, and crossfade into the master.
P1	Waveform proxies	Generate compact peak data and preview audio for responsive browser editing.
P2	Silence and breath tools	Detect long pauses, mouth clicks, breaths, and room-tone gaps with batch suggestions.
P2	Markers and regions	Chapter markers, credits, sample range, QC findings, comments, and custom markers.
P2	Keyboard editing	Spacebar playback, J/K/L transport, zoom, marker navigation, cut, ripple delete, undo, and redo.

5.7 Background adder: music, ambience, and effects

Priority	Capability	Requirement
P1	Asset sources	Upload owned assets or select from an organization library with license and attribution metadata.
P1	Track types	Music, ambience, sound effect, room tone, stinger, and transition.
P1	Narration-aware ducking	Sidechain narration into background tracks with adjustable threshold, ratio, attack, release, floor, and look-ahead.
P1	Per-chapter controls	Apply to one chapter, a selected range, a scene, or the entire book.
P1	Loop, trim, fade, and crossfade	Create seamless beds and transitions without destructive edits.
P1	Real-time preview	Preview a short region using proxy renders before committing a full mix.
P1	Presets	Save and reuse genre, corporate, meditation, education, and credits presets.
P2	Scene-aware suggestions	Suggest restrained background assets based on chapter tone, with user approval required.
P2	Mix intelligibility warnings	Flag background masking, excessive overlap, abrupt starts, and clipping.
P2	Whispersync warning	Warn that overlapping music or effects may affect synchronization and that users must possess rights to all assets.

5.8 Audio remastering

Priority	Capability	Requirement
P1	One-click remaster	Analyze source audio and propose a reversible chain with before/after metrics.
P1	Manual chain	High-pass filter, de-noise, de-click, de-plosive, de-esser, de-reverb, EQ, compression, gate or expander, limiter, and gain.
P1	A/B comparison	Loudness-matched original and processed playback with instant switching.
P1	Batch consistency	Match tone and level across chapters while preserving chapter-specific overrides.
P1	Destination profiles	Separate profiles for ACX technical checks, M4B listening, podcast, training, and custom publisher delivery.
P1	Processing report	Store every parameter, software version, measurement, and input/output checksum.
P2	Adaptive noise profiles	Learn room tone from selected regions and apply conservative reduction.
P2	Repair suggestions	Identify likely mouth clicks, hum, clipping, long silences, and inconsistent loudness for review.

5.9 QC, compliance, and rights

Priority	Capability	Requirement
P0	Versioned QC profiles	Requirements are data-driven and dated so destination changes do not require rewriting the pipeline.
P0	ACX technical profile	Validate chapter-per-file, file duration, naming, credits, sample, RMS, peak, noise floor, sample rate, bitrate mode, bitrate, channels, and spacing.
P0	Eligibility disclaimer	State that technical pass does not guarantee marketplace acceptance and that unauthorized AI/TTS may be prohibited.
P0	Rights attestations	Capture manuscript rights, voice consent, music and effect licenses, and reviewer acknowledgements.
P1	Finding severity	Error, warning, information, waived, and not-applicable with human-readable remediation.
P1	Automatic repair	Offer safe fixes for loudness, peaks, sample rate, bitrate, filenames, spacing, and metadata; never silently alter content.
P1	Cross-file consistency	Check channels, sample rate, noise floor, tonal consistency, naming, order, and duplicate files across a book.
P2	Quality history	Compare QC metrics across versions and providers.

5.10 Publish and delivery

Priority	Capability	Requirement
P0	Chapter MP3 package	Export one chapter or section per file with deterministic names and configurable numbering.
P0	Credits generator	Generate editable opening and closing credits from project metadata.
P0	Retail sample	Select or generate a sample within destination limits and validate content restrictions.
P0	M4B and merged MP3	Continue consumer-friendly outputs with chapter markers and cover metadata.
P1	Package manifest	JSON and human-readable manifest with files, durations, checksums, settings, QC version, and approvals.
P1	Cover validation	Dimensions, aspect ratio, format, color profile, file size, and metadata checks using destination profiles.
P1	Delivery links	Expiring review and download links with revocation, watermark options, and delivery receipts.
P2	Destination connectors	API or guided upload where destination terms and APIs permit.

5.11 Collaboration and review

Priority	Capability	Requirement
P1	Time-coded comments	Comment on chapter, waveform time, transcript selection, QC finding, or metadata field.
P1	Review links	Secure browser review for invited users without exposing internal controls.
P1	Approval gates	Script, voice, narration, mix, master, QC, and publish approvals with required roles.
P1	Change requests	Convert comments into assigned tasks with due dates and resolution states.
P1	Notifications	Email and in-app notices for assignments, failures, completed renders, and approvals.
P2	Version compare	Compare transcript, settings, waveform regions, QC metrics, and export files between renders.

5.12 Library, analytics, billing, and administration

Priority	Capability	Requirement
P1	Asset library	Searchable organization library for audio, covers, voices, pronunciations, and presets.

Priority	Capability	Requirement
P1	License records	Store rights holder, license type, allowed territories, expiration, attribution, and source document.
P1	Project cost ledger	Provider charges, storage, processing, cache savings, discounts, and estimated versus actual cost.
P1	Usage dashboards	Characters, audio minutes, render time, failure rate, QC pass rate, cache hit rate, and cost by project/provider.
P1	Plans and entitlements	Concurrency, storage, retention, providers, seats, collaboration, API, and white-label features.
P1	Fine-grained roles	Owner, admin, producer, editor, reviewer, billing, and read-only roles.
P2	Organization policies	Default QC, retention, approved providers, prohibited features, and required approvals.
P2	Audit explorer	Search security, content, billing, review, delivery, and MCP/API events.

5.13 API, webhooks, MCP, and ecosystem

Priority	Capability	Requirement
P1	Versioned REST API	Stable resource APIs with idempotency keys, cursor pagination, consistent errors, and OpenAPI documentation.
P1	Webhooks	Signed deliveries for job, render, review, QC, export, usage, and billing events with retries and logs.
P1	Scoped service accounts	Organization and project scopes, expiration, rotation, and least privilege.
P2	MCP write tools	Create projects, start scoped renders, cancel jobs, request QC, and create review links with audit events.
P2	Batch CLI	Headless import, render, QC, and export for studios and CI pipelines.
P3	Plugin processing SDK	Allow approved audio analysis or processing modules to run in isolated workers.

6. UI and Interaction Specification

6.1 Product shell

The board establishes the visual direction: a dark, professional production interface with strong stage navigation, waveform-centric editing, and distinct builder, background, and remaster workspaces. The production app should support both light and dark themes, but the dark studio theme is the default for long audio-editing sessions.

Region	Behavior
Left navigation	Collapsible global navigation; active organization selector; project, jobs, library, reviews, analytics, team, billing, settings.
Project header	Breadcrumb, project title, stage status, save state, collaborators, version selector, help, and primary next action.
Stage rail	Import, Script, Voices, Build, Edit, Background, Remaster, QC, Publish. Completed, active, blocked, and warning states.
Main canvas	Context-specific editing workspace with persistent selection and undo/redo.
Inspector	Right-side settings and metadata for the current chapter, segment, clip, track, finding, or export.
Bottom transport	Play/pause, time, chapter navigation, playback speed, loop, A/B, volume, and keyboard shortcut help.
Job drawer	Global live job progress, cancel, retry, logs, and background completion notifications.

6.2 Required routes

Route	Screen
/dashboard	Portfolio dashboard and recommended next actions
/projects	Project library
/projects/new	New project import wizard
/projects/:projectId/overview	Project checklist, recent renders, blockers, and activity
/projects/:projectId/script	Chapter and narration script editor
/projects/:projectId/voices	Voice studio, casting, and pronunciations
/projects/:projectId/build	Preflight estimate, build scope, and live progress
/projects/:projectId/edit	Narration waveform and transcript timeline
/projects/:projectId/background	Music, ambience, effects, ducking, and mix preview
/projects/:projectId/remaster	Analysis, one-click remaster, manual chain, and A/B
/projects/:projectId/qc	Destination profile, findings, auto-fixes, waivers, and report
/projects/:projectId/publish	Credits, sample, metadata, cover, exports, and delivery
/jobs	Organization job queue and history
/library	Assets, voices, pronunciations, and presets
/reviews	All items awaiting review or approval
/analytics	Usage, cost, quality, and throughput
/team	Members, invitations, roles, and service accounts
/settings/*	Organization, providers, storage, billing, notifications, integrations, policies

6.3 Screen specifications

Project library

- Cards and table view with cover, title, author, stage, progress, last activity, assigned producer, QC state, duration, and spend.
- Search by title, author, tag, project identifier, narrator, or client; filters for stage, owner, status, provider, and date.
- Primary actions: New project, import template, duplicate, archive, restore, and bulk assign.
- Empty state includes sample project and a guided import path, not a blank screen.

Script and chapters

- Three-pane layout: chapter tree, editable narration script, and inspector for chapter metadata, voice, pacing, and notes.
- Detected non-narrated matter is highlighted and can be include, exclude, or transform.
- Inline pronunciation candidates and validation messages do not block typing.
- Every structural operation shows its impact on existing renders and asks whether to invalidate dependent artifacts.

Voice studio

- Catalog cards include provider, locale, gender, style, cost, availability, commercial-use status, favorite, and preview.
- A casting table maps default narrator, chapters, characters, and dialogue selections to voice profiles.
- Preview queue compares several voices using the same excerpt and loudness-matched playback.
- Pronunciation drawer supports plain replacement, IPA or phoneme, provider syntax, and test rendering.

Build center

- Preflight summary shows changed chapters, segments to synthesize, cache reuse, cost, estimated duration, outputs, and policy warnings.
- Build scope supports entire book, changed only, selected chapters, failed only, or preview range.
- Live step timeline uses canonical statuses and displays worker, attempt, start time, estimated completion, and retry reason.
- A job can finish as needs review and must show review actions rather than appearing completed or failed.

Background workspace

- Waveform header and chapter selector remain visible while the asset library and mix inspector occupy two lower panes.
- Drag assets onto music, ambience, or effects tracks; every asset shows license state and source.
- Ducking has simple mode and advanced controls. Simple mode exposes strength only; advanced mode exposes full dynamics parameters.
- Preview Mix renders a bounded selection quickly; Apply and Next saves settings and advances to the next chapter with pending mix work.

Remaster workspace

- Analysis card displays RMS, integrated loudness, sample peak, true peak, noise floor estimate, clipping, silence, sample rate, channels, and bitrate.
- One-click remaster explains each proposed processor and expected result. Users may disable individual processors before apply.
- Manual controls use a reorderable processing chain with presets, bypass, per-stage meters, and reset.
- A/B playback is loudness matched to prevent louder from being mistaken for better.

QC and publish

- QC findings group by book-wide, chapter, content, file, and metadata issues with direct navigation to the repair location.
- Automatic fixes are previewed and listed before they create a new render version.
- Publish screen presents chapter order, credits, sample, cover, metadata, destination profiles, and package manifest in one checklist.
- The final action creates an immutable export package; later changes create a new package version rather than modifying delivered files.

6.4 UI state requirements

State	Required behavior
Loading	Skeletons retain layout; avoid page-wide spinners except initial authentication.
Saving	Show saving, saved timestamp, offline queue, conflict, and retry states.
Empty	Explain the next useful action and provide a sample or template.
Error	Display user-safe message, request identifier, retry, and diagnostic details for authorized roles.
Permission denied	Explain the required role without revealing cross-tenant resource existence.
Provider unavailable	Show fallback options, affected chapters, and cost differences.
Needs review	Show reasons, preview/download, approve, reject, and repair actions.
Stale render	Explain which edits invalidated the artifact and offer selective rebuild.
Offline or network loss	Preserve edits locally, stop destructive actions, and reconcile on reconnect.

6.5 Accessibility and responsive behavior

- Meet WCAG 2.2 AA for contrast, focus visibility, labels, landmarks, error messages, and reduced motion.
- All essential script, review, build, QC, and publish actions must be keyboard-operable.
- Waveform interactions require transcript and list equivalents for screen-reader and keyboard users.
- Desktop is required for full editing; tablet supports review and light edits; mobile supports monitoring, comments, approvals, and playback.
- Never encode job or QC state only by color; pair color with icon and text.

7. Target Technical Architecture

7.1 Architecture decision

Preserve the foundation: Keep Flask, SQLAlchemy, Alembic, Postgres, the durable database queue, FFmpeg, storage adapters, React/Vite, the Next.js ops dashboard, and the provider interface. Add modular domains and versioned contracts around them. Do not introduce Redis or Celery merely to imitate a conventional stack.

Layer	Target design
Web app	React/Vite routed application with query caching, project state, typed API client, design tokens, waveform proxies, and background job drawer.
Ops dashboard	Next.js operator console with separate admin authorization and shared API types or generated OpenAPI client.
API	Flask application factory with blueprints by domain: projects, manuscripts, voices, renders, assets, timelines, QC, exports, reviews, billing, admin.
Database	Postgres remains the system of record. Use normalized core entities plus JSONB for versioned processor settings and manifests.
Queue	Job and JobStep rows with SKIP LOCKED claiming, leases, retries, priorities, dependencies, idempotency, and per-step artifacts.
Workers	Specialized worker capabilities: ingest, TTS, waveform, mix, remaster, QC, export. A worker advertises capabilities and claims compatible steps.
Audio engine	FFmpeg-first processing with adapter interfaces for analyzers and optional ML repair services. Preserve immutable originals.
Storage	S3-compatible object storage for originals, normalized text, speech segments, proxies, stems, masters, reports, exports, and licenses.
Events	Transactional outbox emits domain events to webhooks, notifications, analytics, and SSE streams.
Auth	Supabase or legacy verification remains behind one identity layer; add project roles, service accounts, scopes, and audit events.

7.2 Pipeline v2

Step	Inputs	Outputs	Idempotency key
ingest	Source asset and ingester version	Normalized text, extraction report, page map	source checksum + ingester version
structure	Normalized text and chapter rules	Chapter revision set and segment map	text checksum + rule version
prepare speech	Speakable segment and pronunciation set	Provider payload or SSML	segment checksum + lexicon + settings
synthesize or ingest	Provider payload or uploaded source audio	Immutable narration segment asset	provider + voice + payload + settings
align	Segment audio and transcript	Sentence or word time map	audio checksum + alignment version
assemble narration	Segment assets and edit decision list	Chapter narration stem	ordered segment hashes + edit list
mix	Narration stem, background clips, automation, ducking preset	Chapter mix stem	timeline version + mix preset
master	Mix stem and processing chain	Mastered chapter asset and measurements	input checksum + chain version
QC	Mastered chapters, metadata, destination profile	QC report and findings	asset checksums + profile version

Step	Inputs	Outputs	Idempotency key
export	Approved masters, metadata, cover, package profile	Files, manifest, checksums, signed delivery	approved render + package profile

7.3 Audio engineering requirements

- **Measurement:** Use fprobe and FFmpeg analysis to capture sample rate, channels, codec, bitrate, duration, peak, RMS, loudness, true peak, silence, and estimated noise floor. Store raw and rounded values.
- **Mastering:** Represent each processing chain as ordered, versioned processor nodes. Processing is deterministic from an immutable input, chain, and engine version.
- **Background mixing:** Use separate narration, music, ambience, effects, and room-tone buses. Render with sidechain ducking and explicit automation curves.
- **Preview:** Generate short proxy renders for selected ranges. Never wait for a full-book render to hear a change.
- **Consistency:** Analyze cross-chapter tone and level and flag outliers rather than blindly forcing every chapter to identical statistics.
- **Output profiles:** Treat ACX chapter MP3 delivery and consumer M4B delivery as separate outputs with separate encoding and QC profiles.
- **Original safety:** Source audio and TTS segment assets are immutable. Edits and processing always create derived artifacts.

7.4 Security and governance architecture

Control	Requirement
Authorization	Every resource carries organization_id; project-level access is evaluated through roles and explicit shares. Cross-org remains indistinguishable from not found.
Service accounts	Hashed tokens, organization and project scopes, expiration, rotation, last-used timestamp, IP restrictions where needed.
Audit events	Immutable records for auth, membership, source replacement, render, approval, export, delivery, license, billing, admin, API, and MCP actions.
Signed access	Short-lived uploads and downloads, revocable review links, download limits, and optional email verification.
Secrets	No provider secrets in project settings or logs; use environment or managed secret storage and redact structured logs.
Media rights	License and consent records are attached to assets and checked at publish time.
CORS and browser security	Reject wildcard credentialed CORS in production, use a strict allowlist, CSP, secure cookies where used, and CSRF protection for cookie flows.
Data lifecycle	Configurable retention by artifact class, legal hold, project archive, secure deletion queue, and deletion evidence.

8. Data Model and API Plan

8.1 Core data model changes

Entity	Purpose	Important fields
Project	Persistent book workspace	id, organization_id, title, author, status, default_language, active_revision_id, template_id, metadata_json, created_by
SourceAsset	Original manuscript or narration source	project_id, asset_id, source_type, checksum, ingester_version, extraction_status, provenance
ProjectRevision	Immutable source/script snapshot	project_id, version, parent_id, status, source_asset_id, normalized_text_key, created_by, approved_at
Chapter	Stable chapter identity across revisions	project_id, stable_key, order_index, kind, current_revision_id
ChapterRevision	Versioned chapter script	revision_id, chapter_id, title, source_text, speakable_text, checksum, estimated_duration
NarrationSegment	Smallest renderable unit	chapter_revision_id, order_index, text, checksum, voice_assignment_id, performance_json
VoiceProfile	Reusable voice configuration	organization_id, provider, voice_id, engine, locale, settings_json, consent_record_id
VoiceAssignment	Maps a voice to project scope	project_id, chapter_id, character_key, segment_selector, voice_profile_id, priority
PronunciationRule	Lexicon entry	scope_type, scope_id, token, replacement, phoneme, provider_payloads_json, enabled
Job	Asynchronous operation	job_type, project_id, input_revision_id, render_id, status, priority, settings_json, idempotency_key
JobStep	Checkpointed pipeline step	job_id, step_type, dependency_ids, status, attempts, worker_capability, input_hash, output_manifest
AudioAsset	Immutable stored audio	organization_id, project_id, kind, storage_key, checksum, duration, codec, sample_rate, channels, source_asset_id
WaveformProxy	Browser preview data	audio_asset_id, peaks_key, preview_key, resolution, version
Timeline	Non-destructive edit decision list	project_id, chapter_id, version, sample_rate, duration, created_by
TimelineTrack	Narration or background bus	timeline_id, type, name, order_index, mute, solo, gain
TimelineClip	Placement of an asset	track_id, audio_asset_id, start, source_in, source_out, gain, fades_json, automation_json
ProcessingPreset	Mix, master, or QC settings	organization_id, type, name, version, settings_json, locked, created_by
Render	Versioned production result	project_id, revision_id, timeline_version, status, narration_asset_id, mix_asset_id, master_asset_id, manifest_key
QCProfile	Versioned destination specification	slug, version, effective_date, rules_json, source_url, active
QCReport	Evaluation of a render or package	render_id, profile_id, status, metrics_json, report_key, created_at
QCFinding	Actionable issue	report_id, severity, rule_code, chapter_id, time_range, message, remediation, state, waived_by
ReviewThread	Review context	project_id, render_id, scope_type, scope_id, status, assigned_to

Entity	Purpose	Important fields
ReviewComment	Time-coded or field comment	thread_id, author_id, body, time_range, text_range, resolved_at
Approval	Formal gate decision	project_id, render_id, gate, status, reviewer_id, decision_note, decided_at
ExportPackage	Immutable delivery set	project_id, render_id, profile, version, status, storage_prefix, manifest_key, delivered_at
LicenseRecord	Rights for media or voices	asset_id, rights_holder, license_type, territories, starts_at, expires_at, proof_key
AuditEvent	Security and business audit trail	organization_id, actor_type, actor_id, action, resource_type, resource_id, metadata_json, occurred_at
WebhookEndpoint	External event subscription	organization_id, url, secret_hash, event_types, enabled
WebhookDelivery	Signed delivery attempts	endpoint_id, event_id, status, attempts, response_code, next_attempt_at

8.2 API standards

- Introduce /api/v1 while preserving existing routes as deprecated adapters until the frontend migration is complete.
- Use cursor pagination for collection endpoints and include total estimates only where efficient.
- Require Idempotency-Key for create-render, create-export, and other costly mutations.
- Return one error envelope: code, message, details, request_id, and optional remediation.
- Generate OpenAPI definitions and TypeScript clients; remove hand-maintained divergence between backend and frontend status types.
- Use ETags or revision numbers for optimistic concurrency on script, timeline, preset, and metadata updates.
- Use SSE for job and review events with polling fallback.

8.3 Proposed endpoint groups

Group	Representative endpoints
Projects	GET/POST /api/v1/projects; GET/PATCH/DELETE /projects/:id; POST /projects/:id/duplicate; POST /projects/:id/archive
Sources and revisions	POST /projects/:id/sources; GET /projects/:id/revisions; POST /projects/:id/revisions; POST /revisions/:id/approve
Chapters and script	GET /revisions/:id/chapters; PATCH /chapter-revisions/:id; POST split, merge, reorder, transform, lint
Voices and pronunciation	GET /voices; POST /voice-previews; CRUD /voice-profiles; CRUD /pronunciations; PUT /voice-assignments
Jobs and renders	POST /projects/:id/renders; GET /jobs/:id; GET /jobs/:id/events; POST cancel, retry, approve, reject
Timeline and assets	CRUD /assets; GET/PUT /chapters/:id/timeline; POST /timeline/previews; POST /pickups
Background and remaster	POST /mix-previews; CRUD /mix-presets; POST /remaster-previews; CRUD /master-presets
QC	GET /qc-profiles; POST /renders/:id/qc; GET /qc-reports/:id; POST /findings/:id/fix or waive
Review	POST /review-links; CRUD /review-threads and /comments; POST /approvals

Group	Representative endpoints
Publish	POST /export-packages; GET /export-packages/:id; POST /deliveries; POST revoke
Usage and billing	GET /usage; GET /costs; GET /entitlements; POST /billing/checkout and /portal
Administration	GET /admin/workers, storage, providers, jobs, audits; POST drain worker, retry job, rotate key
Integrations	CRUD /webhooks; GET /webhook-deliveries; CRUD /service-accounts

8.4 Compatibility behavior

Legacy behavior	Migration behavior
POST /api/synthesize creates a project and job together	Adapter creates a one-time project revision and a render job, then returns the legacy shape. New UI uses project and render endpoints.
GET /api/task/:id and frontend started/processing/completed types	Backend remains canonical queued/running/succeeded/needs_review/failed/canceled; legacy translation is temporary and tested.
Project.source_text stores mutable full text	Migrate to source asset and revision records; retain source_text during transition and stop writing it after cutover.
Job output_mp3 and output_m4b paths	Use AudioAsset and ExportPackage; retain key columns as compatibility references until all consumers migrate.

9. Exact Repository File Plan

Paths marked MODIFY exist in the current repository baseline. Paths marked NEW define the target organization. New modules should be introduced incrementally; do not move unrelated files in the same pull request as feature logic.

9.1 Immediate stabilization files

Action	Path	Change
MODIFY	frontend/src/types/index.ts	Replace invented task statuses with canonical backend statuses; include needs_review, canceled, failed chapter state, identifiers, timestamps, and review actions.
MODIFY	frontend/src/App.tsx	Poll queued and running; stop on all terminal states; support needs_review; load existing project/job rather than only in-memory state.
MODIFY	frontend/src/components/ProgressTracker.tsx	Render all canonical states, review actions, cancel, retry, and format downloads.
MODIFY	frontend/src/services/api.ts	Centralize error envelope, typed endpoints, approve/reject/delete/usage, and response validation.
NEW	frontend/src/test/setup.ts	Testing Library and browser API setup.
NEW	frontend/src/components/__tests__/ProgressTracker.test.tsx	State coverage for queued, running, needs_review, succeeded, failed, canceled.

Action	Path	Change
NEW	frontend/src/App.test.tsx	Polling and terminal transition coverage.
MODIFY	.github/workflows/ci.yml	Run frontend and dashboard tests; use npm ci for dashboard; upload test reports.
DELETE	.mcp-write-probe	Remove tracked write probe.
NEW	.github/dependabot.yml	Monthly Python and npm dependency updates grouped by service.
MODIFY	backend/app.py	Reject insecure wildcard credentialed CORS; protect cache stats; add paginated job list adapter.

9.2 Backend target modules

Action	Path	Responsibility
MODIFY	backend/app.py	Convert to application factory and register versioned blueprints; preserve compatibility routes.
NEW	backend/api/v1/projects.py	Project CRUD, archive, duplicate, checklist, activity.
NEW	backend/api/v1/sources.py	Signed uploads, ingestion, source replacement, provenance.
NEW	backend/api/v1/chapters.py	Chapter structure and script operations.
NEW	backend/api/v1/voices.py	Voice catalog, previews, profiles, assignments, pronunciation tests.
NEW	backend/api/v1/renderers.py	Preflight, create render, selective rebuild, status, approvals.
NEW	backend/api/v1/assets.py	Asset CRUD, metadata, license links, signed access.
NEW	backend/api/v1/timelines.py	Timeline versions, clips, tracks, previews, pickups.
NEW	backend/api/v1/qc.py	Profiles, reports, findings, fixes, waivers.
NEW	backend/api/v1/reviews.py	Review links, threads, comments, approvals.
NEW	backend/api/v1/exports.py	Package generation, manifests, deliveries, revocation.
NEW	backend/api/v1/integrations.py	Webhooks and service accounts.
MODIFY	backend/db/models.py	Add project status and compatibility relationships only; move new domains into model modules over time.
NEW	backend/db/models_project.py	SourceAsset, ProjectRevision, Chapter, ChapterRevision, NarrationSegment.
NEW	backend/db/models_audio.py	AudioAsset, WaveformProxy, Timeline, TimelineTrack, TimelineClip, Render.
NEW	backend/db/models_quality.py	ProcessingPreset, QCProfile, QCReport, QCFinding, ExportPackage.
NEW	backend/db/models_collaboration.py	ReviewThread, ReviewComment, Approval, AuditEvent, LicenseRecord.
NEW	backend/db/models_integrations.py	ServiceAccount, WebhookEndpoint, WebhookDelivery, OutboxEvent.

Action	Path	Responsibility
MODIFY	backend/jobs/queue.py	Claim compatible JobStep rows, priorities, dependencies, idempotency, leases, and cancellation propagation.
MODIFY	backend/jobs/pipeline.py	Retain as legacy orchestrator, then delegate to step handlers.
NEW	backend/jobs/orchestrator.py	Build job dependency graph and mark steps ready.
NEW	backend/jobs/steps/*.py	Ingest, structure, synthesize, align, assemble, mix, master, QC, export handlers.
NEW	backend/audio/analysis.py	Measurements and fprobe wrappers.
NEW	backend/audio/timeline.py	Render edit decision lists and automation.
NEW	backend/audio/mix.py	Background buses, ducking, fades, and preview renders.
NEW	backend/audio/master.py	Processing chain, presets, A/B previews, batch consistency.
NEW	backend/audio/qc.py	Profile-driven rule engine and reports.
NEW	backend/audio/export.py	Chapter MP3, M4B, merged MP3, sample, credits, manifests.
MODIFY	backend/services/providers/base.py	Add provider capabilities, preview, SSML, styles, timestamps, and normalized error classes.
NEW	backend/services/ingesters/*.py	DOCX, PDF, TXT, EPUB, Markdown, and optional OCR adapters.
NEW	backend/services/outbox.py	Transactional domain events.
NEW	backend/services/permissions.py	Project roles and policy evaluation.
MODIFY	backend/mcp_server.py	Move to scoped identity and audit before adding write tools.

9.3 Frontend target modules

Action	Path	Responsibility
NEW	frontend/src/app/router.tsx	Route definitions, loaders, error boundaries, protected routes.
NEW	frontend/src/app/AppShell.tsx	Global navigation, organization selector, job drawer, notifications.
NEW	frontend/src/api/generated/*	OpenAPI-generated types and client.
NEW	frontend/src/features/projects/*	Project library, overview, checklist, activity, templates.
NEW	frontend/src/features/script/*	Chapter tree, script editor, lint, structural operations, revisions.
NEW	frontend/src/features/voices/*	Voice catalog, comparison, profiles, casting, pronunciation.
NEW	frontend/src/features/build/*	Preflight, job timeline, retries, selective builds.
NEW	frontend/src/features/timeline/*	Waveform, transcript alignment, tracks, clips, pickups, transport.
NEW	frontend/src/features/background/*	Asset browser, ducking, fades, mix presets, preview.

Action	Path	Responsibility
NEW	frontend/src/features/remaster/*	Analysis, processor chain, presets, A/B comparison.
NEW	frontend/src/features/qc/*	Profiles, findings, navigation, auto-fixes, waivers, reports.
NEW	frontend/src/features/publish/*	Credits, sample, metadata, cover, packages, deliveries.
NEW	frontend/src/features/reviews/*	Review portal, comments, tasks, approvals.
NEW	frontend/src/components/ui/*	Accessible buttons, fields, dialogs, tabs, menus, toasts, data tables.
NEW	frontend/src/components/audio/*	Waveform, transport, meters, A/B player, region selector.
NEW	frontend/src/lib/jobEvents.ts	SSE subscription with polling fallback.
NEW	frontend/e2e/*.spec.ts	Playwright critical journeys.

9.4 Dashboard and operations files

Action	Path	Responsibility
MODIFY	dashboard/app/dashboard/page.tsx	Add worker capacity, storage, delivery, and trend summaries.
NEW	dashboard/app/dashboard/workers/page.tsx	Workers, capabilities, leases, draining, saturation.
NEW	dashboard/app/dashboard/jobs/page.tsx	Cross-org queue, attempts, step graph, retry and cancellation.
NEW	dashboard/app/dashboard/providers/page.tsx	Availability, latency, error rate, fallback, cost.
NEW	dashboard/app/dashboard/storage/page.tsx	Bytes, orphan assets, retention queue, failed deletions.
NEW	dashboard/app/dashboard/deliveries/page.tsx	Webhook and export delivery attempts.
NEW	dashboard/app/dashboard/audit/page.tsx	Security and administrative event explorer.
NEW	dashboard/tests/*	Component and route tests.

10. Delivery Roadmap and Pull Request Sequence

10.1 Team and planning assumptions

- Core team: two full-stack engineers, one audio or backend engineer, one product designer, and one QA engineer at least half time, with shared DevOps and security review.
- A single AI agent may prepare code and tests, but every migration, auth change, billing change, audio standard, and destructive operation receives human review.
- Each pull request is independently deployable or hidden behind a feature flag. Database migrations are backward compatible until the final cleanup release.
- No pull request combines a broad file move with new behavior. Refactoring follows or precedes feature work with explicit tests.

10.2 Phase roadmap

Phase	Weeks	Deliverables	Exit gate
0. Stabilize	1-2	Status contract, frontend tests, CI test gates, CORS/cache hardening, cleanup, dependency automation	Current build completes and downloads correctly through the UI; required checks are green
1. Workspace foundation	3-6	Project CRUD, source assets, revisions, chapters, migration path, routed shell, project library	A project persists across sessions and a manuscript can be revised without losing history
2. Builder core	7-10	Voice profiles, pronunciation, preflight, selective builds, job steps, live progress, preview	Builder MVP usable by pilot authors on real books
3. Timeline and background	11-15	Audio assets, waveform proxies, transcript alignment, edit decision lists, background tracks, ducking, mix previews	A reviewer can edit a chapter and approve a narration-plus-background mix
4. Remaster and QC	16-20	Processing chains, one-click remaster, manual controls, A/B, destination profiles, ACX technical report	Pro Beta produces reproducible masters and complete QC findings
5. Collaboration and publish	21-24	Review links, comments, approvals, credits, sample, cover checks, chapter packages, manifests, delivery links	A complete project can move through approval to immutable export package
6. Scale and monetize	25-28	Billing, entitlements, analytics, scoped service accounts, webhooks, MCP writes, admin depth, performance and security hardening	GA operational readiness review passes

10.3 Pull request sequence

PR	Title	Scope
PR 0	fix(frontend): align canonical job states and terminal behavior	P0 status types, polling, ProgressTracker, review actions, tests
PR 1	chore(ci): enforce frontend and dashboard test gates	Vitest/RTL setup, Playwright smoke, npm ci, Dependabot, probe deletion
PR 2	feat(projects): persistent project workspace and source assets	Project status, SourceAsset, storage, project CRUD, migration
PR 3	feat(manuscripts): revisions, chapters, and speakable script	ProjectRevision, Chapter, ChapterRevision, editor APIs, ingestion report
PR 4	feat(ui): routed application shell and project library	Router, AppShell, design tokens, project screens, feature flags
PR 5	feat(voices): voice profiles, previews, and pronunciations	Provider capabilities, preview cache, profiles, assignments, lexicon
PR 6	feat(jobs): durable job steps and selective render planning	JobStep, orchestrator, idempotency, step claims, SSE events
PR 7	feat(builder): preflight and changed-only builds	Cost and duration estimate, build scope, live step timeline, retry UX
PR 8	feat(audio): assets, waveform proxies, and timelines	AudioAsset, WaveformProxy, Timeline entities, browser waveform and transport

PR	Title	Scope
PR 9	feat(background): multi-track mixer and narration ducking	Library assets, clips, buses, preview render, presets, license warnings
PR 10	feat(remaster): profile-driven mastering and A/B review	Analysis metrics, processor chain, previews, presets, batch consistency
PR 11	feat(qc): versioned destination profiles and findings	QCProfile, reports, rules, auto-fix queue, ACX technical profile
PR 12	feat(review): comments, approvals, and secure review links	Review portal, time-coded notes, tasks, stage gates
PR 13	feat(publish): credits, samples, chapter exports, and manifests	ExportPackage, deterministic filenames, cover validation, delivery links
PR 14	feat(billing): entitlements, cost ledger, and subscriptions	Plans, usage dimensions, checkout, portal, limits, invoices
PR 15	feat(integrations): webhooks, service accounts, and scoped MCP writes	Outbox, signatures, scopes, audit, agent tools
PR 16	perf(security): GA hardening and operational readiness	Load tests, worker scaling, storage lifecycle, backup restore, penetration review, runbooks

10.4 Mandatory PR template

- Problem statement and user outcome.
- Exact in-scope and out-of-scope list.
- Database migration and rollback or compatibility plan.
- API contract changes and generated client update.
- Security and multi-tenant review.
- Audio fixture or visual UI evidence where applicable.
- Unit, integration, end-to-end, migration, and accessibility tests.
- Observability, dashboards, alerts, and runbook changes.
- Feature flag and rollout plan.
- Post-deploy verification and rollback trigger.

11. Testing, Security, Operations, and Release Management

11.1 Test pyramid

Layer	Required coverage
Backend unit	Text transforms, rules, cost estimation, permissions, idempotency, manifests, status transitions, processors, filename rules.
Backend integration	Postgres migrations, SKIP LOCKED concurrency, step dependencies, retries, cancellation, storage, provider stubs, outbox, webhooks.
Audio golden tests	Small licensed or generated fixtures with expected duration, codec, sample rate, bitrate, channels, RMS, peaks, noise floor estimates, and checksums where deterministic.
Frontend unit	Forms, editors, canonical job states, QC finding actions, review permissions, errors, autosave, conflict states.
Frontend integration	API mocking for project, build, mix, remaster, QC, review, and publish workflows.

Layer	Required coverage
End-to-end	Sign up or login, create project, upload, edit chapter, select voice, build preview, mix, remaster, QC, approve, export, download.
Security	Cross-org attempts, role matrix, service scopes, signed links, revocation, CORS, rate limits, malicious filenames, media validation, webhook signatures.
Performance	Project lists, large manuscripts, 100+ chapter books, concurrent workers, waveform proxies, preview render latency, storage throughput.
Recovery	Worker crash at every step, database failover, object storage outage, provider outage, partial upload, webhook retry, restore from backup.

11.2 Release gates

Gate	Requirement
Code quality	Lint, type check, unit, integration, migration, frontend, dashboard, and end-to-end checks pass.
Data safety	Backward-compatible migration tested on a production-like database snapshot; rollback or forward-fix procedure documented.
Tenant safety	Automated cross-organization and role-matrix tests pass for every new resource.
Audio quality	Golden fixtures and fprobe assertions pass; visual or audible A/B evidence attached for processing changes.
Accessibility	Automated checks plus keyboard and screen-reader smoke test for changed journeys.
Observability	Metrics, structured logs, request and job correlation, alert, and runbook are present.
Product acceptance	Acceptance criteria signed off in staging with realistic project data.
Rollout	Feature flag, cohort, monitoring window, rollback trigger, and owner defined.

11.3 Operational metrics and alerts

Signal	Target or alert
API availability	Target 99.5% during beta and 99.9% after GA scale validation.
Job start latency	p95 queued-to-running under 60 seconds when capacity is available.
Job failure rate	Under 2% excluding invalid user inputs and deliberate provider outages.
Orphan recovery	All stale jobs detected within lease plus sweep interval; no job remains running indefinitely.
Preview latency	Selection preview begins within 10 seconds for cached or short content under normal load.

Signal	Target or alert
Cache hit rate	Track by provider and project; alert on sudden collapse after releases.
QC pass rate	Track first-pass and final-pass separately; target 98% final technical pass for supported profiles.
Storage errors	Alert on failed puts, failed signed URLs, missing referenced objects, and retention deletion failures.
Provider health	Availability, p50/p95 latency, errors, throttling, and spend by provider.
Webhook delivery	Alert when an endpoint exceeds retry threshold or signing failures occur.

11.4 Deployment model

- Near term Railway topology may continue combining API and worker when local storage requires one mounted volume, but production scale should move all durable media to S3-compatible storage so workers can scale independently.
- Migrations run as a dedicated release step or one controlled instance, not concurrently in every scaled process.
- Workers advertise capabilities and may be deployed separately for TTS, audio processing, and export when resource profiles diverge.
- Use blue-green or rolling deployments with old and new schema compatibility across the rollout window.
- Maintain daily database backups, object storage versioning or lifecycle protection, quarterly restore drills, and documented recovery objectives.

12. Metrics, Risks, Governance, and Definition of Done

12.1 Product metrics

Metric	Definition	Initial target
Activation	User creates a project and produces a playable first chapter	60% of new organizations within 24 hours
Time to first preview	Import complete to first playable project-specific voice preview	Under 10 minutes median
Selective rebuild rate	Corrections that avoid full-book synthesis	At least 90%
Final QC technical pass	Approved export packages without blocking technical findings	98%
Job failure rate	Failed jobs divided by started jobs, excluding user cancellation	Under 2%
Review turnaround	Time from review request to approval or change request	Track by customer segment
Cache savings	Estimated provider cost avoided through reuse	Visible per project and organization
Retention	Organizations producing or updating another project within 60 days	Set after beta cohort baseline

12.2 Principal risks and mitigations

Risk	Likelihood / impact	Mitigation
Edge TTS endpoint changes or closes	Medium / High	Capability health checks, provider fallback, enterprise providers, transparent cost estimates, no single-provider promises.
ACX or retailer policy changes	High / High	Versioned destination profiles with effective dates, official source links, eligibility disclaimers, regular policy review.
Unauthorized voice or media use	Medium / High	Consent and license records, restricted features, publish-time rights checks, audit trail, takedown and revocation workflow.
Audio processing damages quality	Medium / High	Non-destructive originals, conservative defaults, A/B, loudness matching, golden tests, human approval.
Frontend complexity grows faster than tests	High / High	Feature-folder architecture, generated API types, required unit and E2E gates from Phase 0.
Schema expansion slows delivery	Medium / Medium	Introduce domains phase by phase, backward-compatible migrations, avoid premature table moves.
Large files or books overwhelm browser and workers	Medium / High	Multipart uploads, streaming, proxies, cursor pagination, bounded previews, capability workers, load tests.
MCP or API write actions cross tenant boundaries	Medium / Critical	Scoped service identities, policy checks, idempotency, approvals for destructive actions, audit events, security tests.
Background audio distracts or breaks sync features	Medium / Medium	Restrained presets, intelligibility checks, rights warnings, Whispersync warning, default narration-only profile.
Bot-driven changes lack review ownership	High / Medium	CODEOWNERS, required reviews for auth/billing/migrations/audio profiles, signed releases, change log.

12.3 Governance

- Create CODEOWNERS rules for authentication, billing, database migrations, audio QC profiles, storage, and deployment.
- Require at least one human approval for protected paths and two approvals for auth, billing, or destructive retention changes.
- Maintain architecture decision records for queue evolution, timeline format, mastering engine, distribution profiles, and service-account authentication.
- Review official destination requirements quarterly and whenever a customer reports a policy change.
- Treat standards profiles and rights language as controlled content with owner, source, effective date, and review date.

12.4 Definition of done for a feature

- User-facing acceptance criteria pass in staging.
- Canonical API contract and generated frontend types are updated.
- Authorization and cross-tenant tests exist.
- Database migrations are backward compatible and tested on Postgres.
- All new job steps are idempotent, retryable, cancelable, observable, and restart-safe.
- Source assets remain immutable and derived assets have lineage and checksums.

- Unit, integration, frontend, end-to-end, and audio fixture tests pass as applicable.
- Loading, empty, error, permission, and needs-review UI states are implemented.
- Accessibility and keyboard behavior are verified.
- Metrics, logs, alerts, and runbook changes are complete.
- Documentation, release notes, and feature flag cleanup owner are recorded.

13. First 30 Days: Immediate Execution Plan

Week 1: Repair the current experience

- Merge or supersede the dependency and hygiene work in open PR 3, after confirming the target branch and duplicate edge-tts change.
- Fix the frontend/backend status mismatch and add tests for every status transition.
- Add approve, reject, cancel, delete, usage, and job history actions to the web client.
- Protect cache statistics and fail closed on insecure CORS configuration.
- Delete .mcp-write-probe and add dependency automation.

Week 2: Establish product and test foundations

- Add React Testing Library and Playwright smoke tests and make them required in CI.
- Create a routed application shell behind a feature flag while leaving the current screen available.
- Freeze canonical domain status definitions in an OpenAPI schema or shared generated client.
- Create ADRs for persistent project revisions, JobStep orchestration, audio artifact lineage, and destination profiles.

Week 3: Build the persistent project model

- Add Project status and metadata, SourceAsset, ProjectRevision, Chapter, and ChapterRevision migrations.
- Implement project list/create/read/update/archive APIs with cursor pagination.
- Persist original uploads in object storage and record checksums and extraction versions.
- Backfill current Project.source_text into an initial revision without breaking existing jobs.

Week 4: Deliver the first new workflow

- Ship the project library and project overview.
- Ship chapter list and speakable script editor with autosave and revision conflict handling.
- Create a compatibility adapter so current synthesize requests use the new project revision internally.
- Recruit a small pilot set of authors or internal production users and measure time to first preview, failures, and chapter corrections.

Thirty-day success gate: At day 30, the current synthesis flow must be reliable in the browser, every UI state must be tested, projects must persist across sessions, original manuscripts must be retained with provenance, and the first revision-aware chapter editor must be usable behind a feature flag.

Appendix A. Priority Matrix

Priority	Meaning	Release rule
P0	Stability, security, policy, data integrity, or current-product correctness	Must complete before feature expansion or public beta
P1	Core differentiated product capability	Required for General Availability
P2	Professional depth or scale feature	Complete for GA where noted; otherwise scheduled immediately after GA
P3	Ecosystem or advanced expansion	Requires separate validation and risk review

Appendix B. Event Taxonomy

Domain	Events
Project	project.created, updated, archived, restored, duplicated
Source	source.uploaded, ingestion.started, ingestion.completed, ingestion.failed, revision.created, revision.approved
Script	chapter.created, split, merged, reordered, script.updated, pronunciation.updated
Render	render.requested, job.queued, step.started, step.succeeded, step.failed, job.needs_review, render.approved
Audio	asset.created, timeline.updated, preview.ready, mix.ready, master.ready
QC	qc.started, qc.completed, finding.created, finding.fixed, finding.waived
Review	review.requested, comment.created, changes_requested, gate.approved, gate.rejected
Publish	package.requested, package.ready, delivery.created, delivery.downloaded, delivery.revoked
Billing	usage.recorded, quota.warning, quota.exceeded, subscription.updated, invoice.failed
Security	member.invited, role.changed, service_account.created, token.rotated, access.denied, audit.exported

Appendix C. ACX Technical Profile Baseline

Policy warning: This profile validates technical characteristics only. ACX currently states that unauthorized TTS, AI, or automated recordings are prohibited unless authorized, and a technical pass does not guarantee acceptance.

Rule area	Baseline requirement to encode
Content	Separate opening and closing credits; opening states title, author, and narrator; closing indicates finality.
Sample	Retail sample of five minutes or less and taken from the audiobook.
File structure	One chapter or section per file; opening and closing credits are separate files; each file no longer than 120 minutes.
Naming	Chapter or section title included; standard US alphabetical and numeric characters for filenames.
Consistency	Same channel format across files; consistent sound, tone, noise, spacing, and pronunciation.
Average level	RMS between -23 dB and -18 dB per file.
Peak	Peak lower than -3 dB.
Noise floor	Noise floor no higher than -60 dB RMS.
Room tone	Recommended one to five seconds at the beginning and end; not more than five seconds.
Encoding	192 kbps or higher constant bit rate MP3 at 44.1 kHz.
Music and effects	Use sparingly, do not obscure narration, retain rights, and warn that overlap may affect Whispersync.

Appendix D. Evidence and Source Notes

Repository evidence was reviewed against the main branch at commit `f9a81e54eb7ac8985858b2ca4380e108b0b56d85`. The most load-bearing files are listed below.

Source	Evidence used
README.md	Architecture, features, deployment, endpoints, operations, roadmap.
backend/db/models.py	Organization, project, job, chapter, attempt, usage, and rate-limit data model.
backend/app.py	Current API routes, serializer shapes, upload behavior, project/job creation, downloads, usage, health, and CORS.
backend/jobs/queue.py	Claiming, retries, cancellation, leases, orphan recovery, review transitions.
backend/jobs/pipeline.py	Chapter detection, synthesis cache, normalization, QC, assembly, storage, and checkpointing.
backend/utils/audio_utils.py	Current dBFS normalization, silence/clipping checks, MP3 assembly, and M4B export.
backend/services/text_processor.py	Current chapter heuristics, cleanup, abbreviation expansion, and provider chunking.
backend/services/providers/base.py and registry.py	Current provider capabilities and Edge/Polly registry.
frontend/src/App.tsx	Single-screen workflow and polling logic.
frontend/src/types/index.ts	Frontend status contract mismatch.
frontend/src/components/ProgressTracker.tsx	Current completed-only download and limited state display.
frontend/src/services/api.ts	Current client endpoints and authentication token handling.

Source	Evidence used
.github/workflows/ci.yml and frontend/package.json	Current CI gates and unused Vitest capability.
dashboard/app/dashboard/page.tsx	Current operator overview.
backend/mcp_server.py	Current read-only operator tools and single-key authentication.
GitHub PR 3	Known current frontend status mismatch and environment verification.

Official destination sources

- [ACX audio submission requirements, updated April 15, 2026](#)
- [ACX upload audio files guidance, updated April 30, 2026](#)
- [ACX audio quality tool guidance, updated October 20, 2025](#)
- [ACX music and sound effects guidance, updated June 3, 2025](#)

End of plan.